CLASS XI · COMPUTER SCIENCE

Lecture Notes

Six units, taught as one course: how computers represent decisions, how we design algorithms, how Python carries those ideas into programs, how data becomes a database, and how computing affects study, work, and judgement.

SVG diagrams drawn into the notes | Mathematics set with MathJax | Layouts built with flexbox | Units 1 – 6

What these notes are

These are original lecture notes for the Class XI Computer Science course. They follow the unit outcomes in the supplied chapters: computer systems, computational thinking and algorithms, programming fundamentals, data and analysis, applications and impacts of computing, and digital literacy. Worked examples, diagrams, and exercises are written for this set of notes. They are a teaching condensation, not a copy of the textbook.

How to use a lecture

1. Read the outcome line at the start of the unit so you know what you must be able to do.
2. Work every truth table, trace, and short program with a pencil before you look at the result.
3. Say the diagram aloud: inputs, operation, output. If you cannot narrate it, you do not own it yet.
4. Keep the formula strip and the end-of-unit questions for the week before a test.

Contents

1 Computer Systems — Logic, SDLC, OSI and TCP/IP

2 Computational Thinking and Algorithms — Strategies, sort, search

3 Programming Fundamentals — Python, control, libraries

4 Data and Analysis — Databases, ER models, Access

5 Applications and Impacts — AI, IoT, sources, society

6 Digital Literacy — Inquiry, evidence, artefacts

Answer key — Check questions

Computer Systems

UNIT 1

By the end of this unit you can separate analog from digital representation, build truth tables and logic diagrams, simplify small expressions with a Karnaugh map, explain why a software project follows a life cycle, compare Waterfall with Agile, and trace a message through the OSI and TCP/IP models.

1.1 Discrete quantities and continuous quantities

A discrete quantity is counted in separate steps. A continuous quantity can fall anywhere on a range. Computers that use logic gates live in the discrete world: each signal is treated as one of two symbols.

Discrete — stairs

Number of students, books on a shelf, fingers, bits in a byte. You can list the allowed values. There is no “2.5 students” in a headcount.

Continuous — ramp

Temperature, height, time on a clock, the speed of a car. Between any two readings there is another possible reading.



Figure 1. Counting jumps from value to value. Measuring slides through the values in between.

1.2 Analog and digital signals

An analog signal tracks a physical quantity smoothly. A microphone voltage that follows a voice is analog. A digital signal is sampled into symbols. In the circuits of this unit the symbols are only 0 and 1: LOW and HIGH, off and on, false and true.

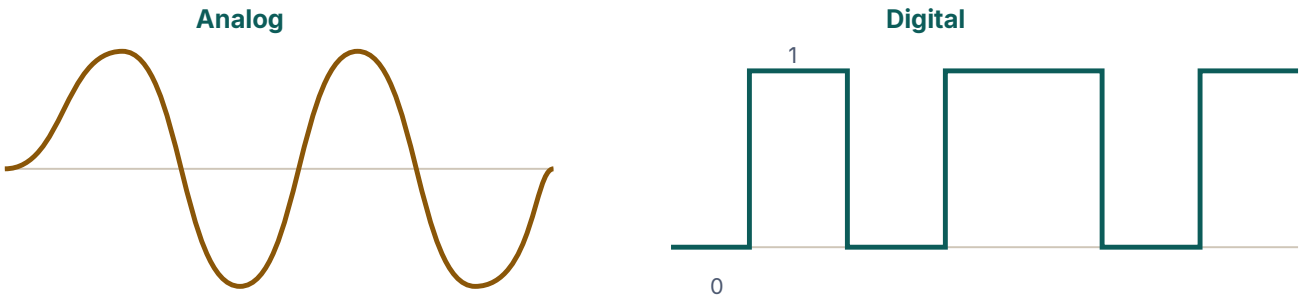


Figure 2. Analog changes through every intermediate level. Digital, in this course, uses two levels only.

Question	Analog	Digital
Values	Any value in a range	A finite set; here, 0 and 1
Noise	A small disturbance changes the value	A level is still read as 0 or 1 if it stays in range
Examples	Mercury thermometer, vinyl groove, dimmer	Keyboard switch, logic gate, stored bit
Processing	Continuous circuits	Boolean operations and logic diagrams

Analog and digital, side by side.

REMEMBER A digital system is still built from the physical world. The *representation* is discrete. The wire voltage may be 0 V or 5 V, and the logic only cares which band it is in.

1.3 Boolean algebra

Boolean algebra is the arithmetic of those two symbols. A variable such as A, B, or Y is either 0 or 1. The three basic operations are AND, OR, and NOT. Several notations appear in books; this course uses the set below.

AND	$Y = A \cdot B$. Also written AB. Output is 1 only when every input is 1.
OR	$Y = A + B$. Output is 1 when at least one input is 1.
NOT	$Y = \overline{A}$. Also written A'. Output is the opposite of the input.
Order	Parentheses, then NOT, then AND, then OR. So $A + B \cdot C = A + (B \cdot C)$, and $\overline{A + B}$ is not the same as $\overline{A} + B$.

Worked evaluation. Let $A = 1$, $B = 1$, $C = 1$, and $Y = A \cdot B + \overline{C}$.

1. NOT first: $\overline{C} = \overline{1} = 0$.
2. AND next: $A \cdot B = 1 \cdot 1 = 1$.
3. OR last: $Y = 1 + 0 = 1$.

De Morgan's laws let you push a bar through a gate and swap the operation. They are the reason a NAND can stand in for other gates.

De Morgan	$\overline{A \cdot B} = \overline{A} + \overline{B}$ and $\overline{A + B} = \overline{A} \cdot \overline{B}$.

1.4 Truth tables

A truth table lists every input combination and the output each one produces. With n inputs there are 2^n rows. Two inputs need 4 rows. Three inputs need 8. Write the inputs in binary counting order so you never skip a row: 00, 01, 10, 11.

A	B	$A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

AND

A	B	$A + B$
0	0	0
0	1	1
1	0	1
1	1	1

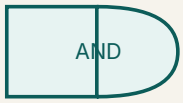
OR

A	$\overline{A}$
0	1
1	0

NOT

1.5 Logic gates

A logic gate is the circuit symbol for one Boolean operation. Each gate has inputs, one output, and a fixed truth table. NAND and NOR are universal: with enough copies of either one, you can build AND, OR, and NOT.



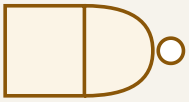
$Y = A \cdot B$. All inputs 1.



$Y = A + B$. Any input 1.



$Y = \overline{A}$. The bubble inverts.



$Y = \overline{A \cdot B}$. AND, then invert.



$Y = \overline{A + B}$. OR, then invert.



$Y = A \oplus B$. 1 when inputs differ.

XNOR is XOR followed by a bubble: $Y = \overline{A \oplus B}$. It outputs 1 when the inputs match. XOR itself is

XOR $A \oplus B = A\overline{B} + \overline{A}B$.

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

Summary of the two-input gates. Memorise the single row that makes each gate special.

WATCH NAND is not “NOT and then AND”. The bubble sits on the output. Invert after the AND. The same order holds for NOR and XNOR.

1.6 From an expression to a table, and back

To table $Y = A \cdot B + C$, add a column for the AND, then a column for the OR. There are $2^3 = 8$ rows.

A	B	C	$A \cdot B$	Y
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	0	1
1	0	0	0	0
1	0	1	0	1
1	1	0	1	1
1	1	1	1	1

To recover a sum-of-products expression, write one minterm for each row where $Y = 1$, then OR those products. A variable that is 0 in that row appears complemented.

Minterms Rows with $Y = 1$ give $\overline{A}\,\overline{B}C + \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$, which simplifies to $C + A \cdot B$.

1.7 Logic diagrams

Draw a gate for each operation, starting with the highest precedence. Wires carry the intermediate results. Do not let two output wires touch.

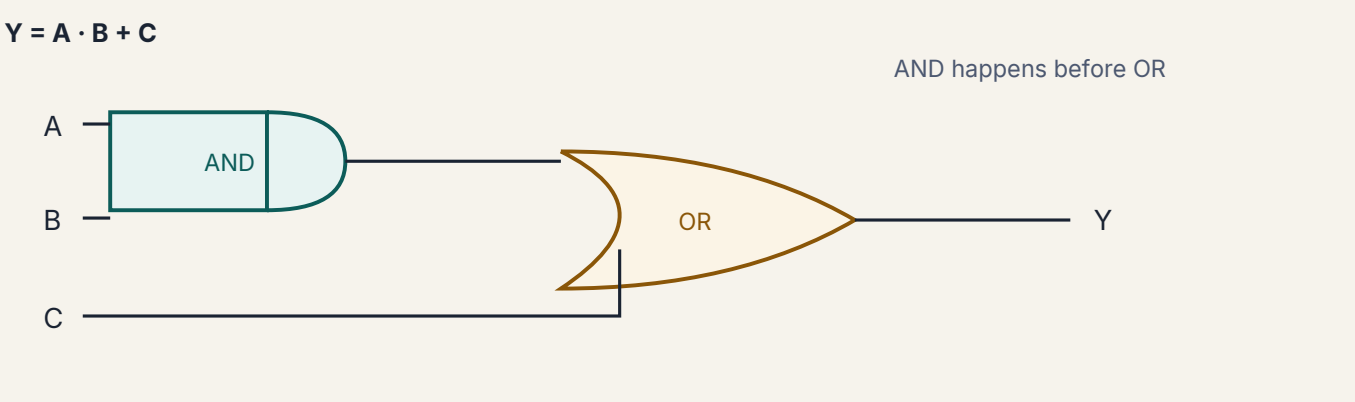


Figure 3. The AND gate owns A and B. Its result meets C at the OR gate.

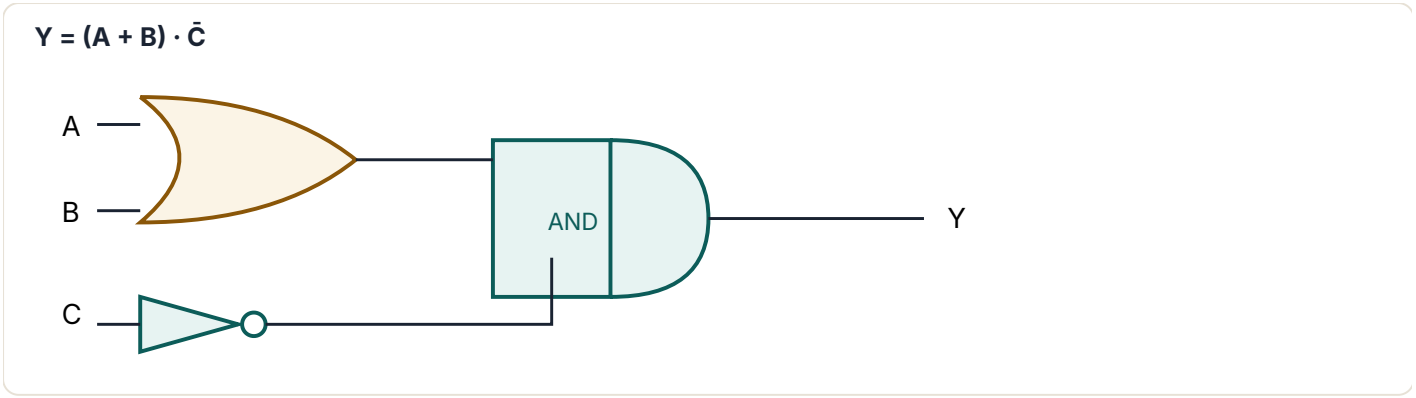
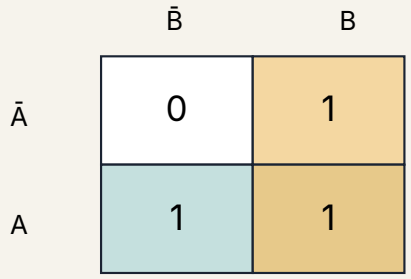


Figure 4. Parentheses force the OR to happen first. C is inverted before it enters the AND.

1.8 Karnaugh maps

A K-map is a truth table redrawn so that neighbours differ by exactly one variable. That neighbour rule is Gray code, not ordinary binary counting. For two variables the column order is $\overline{B}, B$. For three variables the column order is $BC = 00, 01, 11, 10$. You group adjacent 1s in blocks of size $1, 2, 4, 8, \ldots$. Larger groups give simpler terms. Groups may overlap and may wrap around the edges. Each group becomes one product term; the variables that change inside the group are dropped.

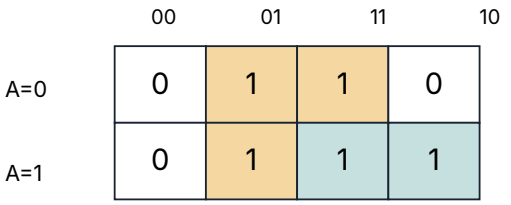
2-variable map



Pair in column B → B. Pair in row A → A. Y = A + B.

Figure 5. Two groups of two. The lone 0 is cell $\overline{A}\,\overline{B}$.

3-variable map



Quad where C = 1 → C. Pair AB (cells 11 and 10 on A=1) → AB. Y = C + A·B. Columns use Gray order, so 11 sits beside 01 and 10.

Figure 6. The quad is every cell with $C = 1$. The pair keeps A and B, and drops C.

TRY Fill a 2-variable map for XOR: outputs 0, 1, 1, 0. Explain why the two 1s cannot form a legal group. The simplified expression stays $A\overline{B} + \overline{A}B$.

1.9 Building the same circuit in Logisim-evolution

Logisim-evolution is the simulation tool for this unit. The habit matters more than any one menu name.

1. Install the Windows build, or the platform-independent JAR if you already have Java. Open a project and save it with a `.circ` name before you draw.
2. From the explorer, open Gates and place a gate. The cursor shows a ghost shape until you click the canvas. Press Escape when you want the select tool back.
3. Input pins are square and can be toggled. Output pins are round and show the result. Place inputs on the left and the output on the right. Double-click a pin to label it.
4. Draw wires from pin to pin. In this tool, wires run horizontally or vertically. A blue wire has no defined value: an input is still floating. A grey wire is not connected at an end. Bright green is logic 1. Dark green is logic 0. Orange or red means a conflict, often two outputs tied together.
5. Change the number of inputs, or the facing direction, from the attribute table while the gate is selected. Simulate and tick through every row of the truth table. For an AND gate the output is bright green only on the row 1, 1.

LAB CHECK Before you trust a circuit, no wire should be blue or grey. Then compare the pin states with a truth table you wrote by hand.

1.10 The software development life cycle

Software is not only source code. It includes the requirements, the design, the tests, the installed product, and the work of keeping it useful. The software development life cycle (SDLC) is the planned path through that work. Skipping a phase usually means paying for it later as confusion, rework, or a system nobody can maintain.

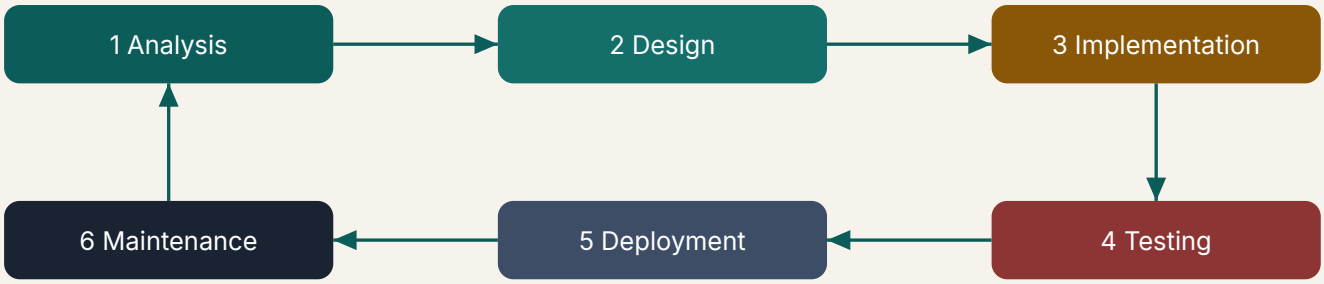


Figure 7. The cycle does not end at installation. Maintenance feeds the next analysis.

Phase	The question it answers	A school-system example
Analysis	What must the software do, and for whom?	Register a book loan, a return, and an overdue notice.
Design	How will data, screens, and modules be shaped?	Tables for books and members; a loan screen; a flowchart of return.
Implementation	How is the design turned into a working build?	Code to the agreed names, comments, and layout.
Testing	Does it meet the requirements?	Unit tests on a fine calculation; a full loan-and-return trial.
Deployment	How does it reach the real users?	Install on the library desk, train the librarian, keep a backup.
Maintenance	How is it corrected, adapted, or improved?	Fix a date bug; add a new report the principal requests.

Black-box testing

You know the required inputs and outputs, not the internal code. A librarian enters a member id and expects the right loan list. Good for acceptance checks.

White-box testing

You can see the code and you choose inputs that walk each branch. Good for the developer who must know that both sides of an `if` have run.

Deployment may be immediate, phased, or parallel with the old process until the new one is trusted. Maintenance is corrective (repair a defect), adaptive (meet a new rule or platform), perfective (make a used feature better), or preventive (reduce a future risk).

1.11 Waterfall and Agile

A life-cycle model is a way of moving through those phases. Waterfall walks them once, in order. A phase is meant to finish, and be signed off, before the next begins. Agile, and the Scrum form of it used in this course, repeats a short cycle so that working software and fresh feedback arrive often.

Waterfall fits when

- The requirements are known and stable.
- The product is like a library system: loans, returns, catalogue, fines.
- A documented hand-off between phases is required.

Strength: a clear plan and an early design. Risk: a change discovered late is expensive, because earlier phases are treated as closed.

Agile fits when

- Stakeholders disagree or discover needs as they try the software.
- The product is like a school mobile app: timetable, results, notices, homework.
- People need a usable slice in weeks, not a finished system in half a year.

Strength: change is expected. Risk: weak discipline turns “flexible” into “unfinished”.

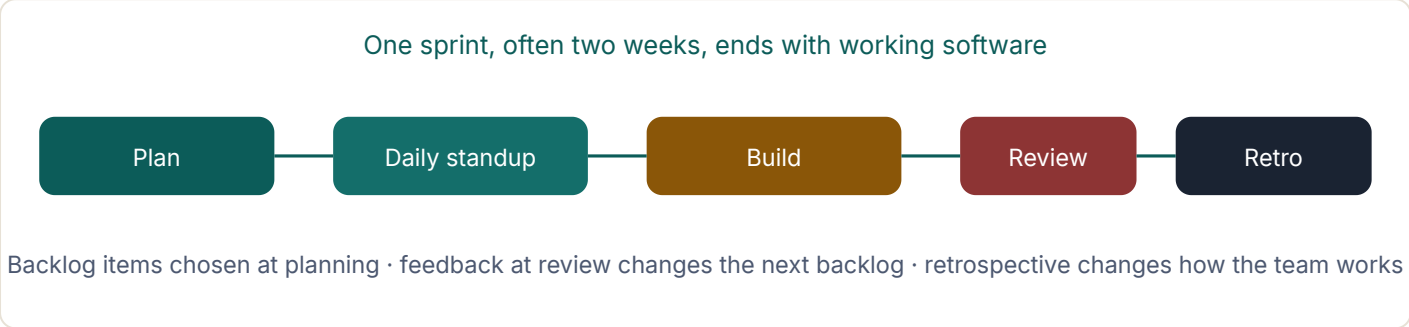


Figure 8. Scrum names you should be able to use: product backlog, sprint, user story, daily standup, sprint review, retrospective.

	Waterfall	Agile (Scrum)
Requirements	Gathered up front	Emerge and get reordered
Delivery	One release after the sequence	A working increment every sprint
Customer role	Reviews documents at phase gates	Tries software and speaks in the review
Change	Formally expensive	Expected between sprints
Best evidence	Stable rules, a schema, a test record	A backlog, sprint summaries, feedback that changed the next sprint

Comparison you can defend with the two case studies: a library system whose rules were known, and a school app whose features kept shifting.

1.12 Communication models

A communication model is a shared picture of how a message moves from one device to another. It divides the job so that each layer can change without rewriting every other layer. The two models in this unit are OSI, which is a teaching and design reference with seven layers, and TCP/IP, which is the working model of the internet with four.

7	Application	HTTP, HTTPS, FTP, SMTP, DNS
6	Presentation	format, compression, encryption
5	Session	open, manage, close a dialogue
4	Transport	TCP, UDP, ports
3	Network	IP, ICMP, routing
2	Data link	MAC, frames, switches
1	Physical	bits, cable, hub, radio

Bottom to top, layers 1 to 7: “Please Do Not Throw Sausage Pizza Away.”

Application, presentation, session

Layer 7 gives the program a network service. Layer 6 translates, compresses, and, in the OSI picture, encrypts. Layer 5 keeps a conversation in order: who is talking, and when the conversation ends. Protocols named with this layer include NetBIOS and RPC.

Transport and network

Layer 4 moves data between processes, using port numbers. Layer 3 moves packets between networks, using logical addresses. IPv4 addresses are 32 bits, as in 192.168.1.1. IPv6 addresses are 128 bits.

Data link and physical

Layer 2 frames the bits, uses MAC addresses, and detects errors on the local link. Its two sublayers are LLC and MAC. Switches and wireless access points live here. Layer 1 moves the raw signal. Hubs and cables do not understand addresses.

	TCP	UDP
Connection	Set up, then transferred, then closed	No handshake
Delivery	Checks, orders, and retransmits	Best effort; a lost datagram stays lost
Use it when	A web page, mail, or file must arrive whole	A live voice or video call prefers a fresh packet to a late one

TCP and UDP share layer 4 and solve different problems.

Encapsulation

On the way down, each layer wraps the unit it received. On the way up, each layer strips the wrapper it understands. That wrapping is encapsulation; the reverse is decapsulation.



Transport adds the segment header · Network adds the packet header · Data link adds the frame · Physical sends bits

Figure 9. Sending travels this way. Receiving walks back from bits to data.

TCP/IP layer	OSI layers it covers	What you should name
Application	5, 6 and 7	HTTP, HTTPS, DNS, SMTP, SSH
Transport	4	TCP and UDP
Internet	3	IP, routing, ICMP
Network access	1 and 2	The local link: MAC, frames, cable or Wi-Fi

TCP/IP folds the OSI layers. Learn the fold, not a second unrelated stack.

WHY BOTH OSI is the precise vocabulary for “which job is this?”. TCP/IP is the model that real internet devices implement. A good answer uses OSI names and then says where that job sits in TCP/IP.

Check questions — Unit 1

1. A classroom thermometer and a count of students: which quantity is continuous, and why?
2. Evaluate $Y = \overline{A + B} \cdot C$ for $A = 1, B = 0, C = 1$. Show the order of operations.
3. A gate outputs 0 only when both inputs are 1. Name it, and write the expression.
4. Why is the column order of a 3-variable K-map 00, 01, 11, 10 rather than 00, 01, 10, 11?
5. A school can describe every screen of a library system before coding starts. A different school wants a phone app and expects parents to change their minds after the first demo. Which model fits each project?
6. A video call drops a late packet instead of waiting. Which transport protocol is the usual choice, and which OSI layer is that?

Computational Thinking and Algorithms

UNIT 2

By the end of this unit you can take a messy problem apart, name an algorithm’s properties, write the idea in steps and in pseudocode, trace bubble sort, selection sort, linear search, and binary search, and choose among them using correctness, efficiency, and clarity.

2.1 Four moves, then an algorithm

Computational thinking is a way of working on a problem so that a computer could later carry it out. The course uses four moves. An algorithm is the result of those moves: a finite list of precise steps that produces the required output from the allowed input, and that does not depend on any one programming language.

Decomposition

Split the problem until each piece can be solved, tested, and reused. Learning a language splits into vocabulary, word order, tense, sentences, listening, and feedback. A program splits the same way: input, process, output, and the errors between them.

Pattern recognition

Ask what repeats, what changes in a regular way, and what can be generalised. A triangle of stars grows by one star each row. That pattern is a loop, not twenty separate print statements.

Abstraction

Keep the steps that define the task. Park the details that do not. Making tea, in every kitchen, is: boil water, add tea, steep, pour, serve. Brand of kettle and colour of cup are noise. Sugar and milk are parameters, not a different algorithm.

Algorithm design

Write the remaining steps so a stranger can follow them. If two careful readers can disagree about the next action, the step is not definite yet.

Star rows

Row i has i stars. After n rows the total is $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$. For $n = 4$, the picture uses 10 stars. Counting the pattern is already an algorithm.

2.2 Algorithm, pseudocode, and the properties of a good one

Algorithm

The logical procedure. Numbered steps or plain sentences. Written for a person who must understand the idea, including someone who does not program. No required keywords.

Pseudocode

The same procedure in a structured sketch: IF, ELSE, FOR, WHILE, and indentation. It is not executed. It is the bridge you cross just before real code. Keywords are conventional, not a secret syntax.

When a question asks for the properties of a good algorithm, list these five, then judge the result with the three criteria in section 2.6.

Property	Meaning in practice
Input	The data the steps are allowed to see is stated. Zero inputs is allowed; an undeclared input is not.
Output	At least one result is produced: a value, a rearranged list, or a clear “not found”.
Definiteness	Each step has one meaning. “Sort the names nicely” fails. “Swap if the left value is greater” holds.
Finiteness	The steps end. A loop needs a condition that becomes false for every legal input.
Effectiveness	Each step is basic enough to do. “Find the best route” is a new problem. “Compare these two numbers” is a step.

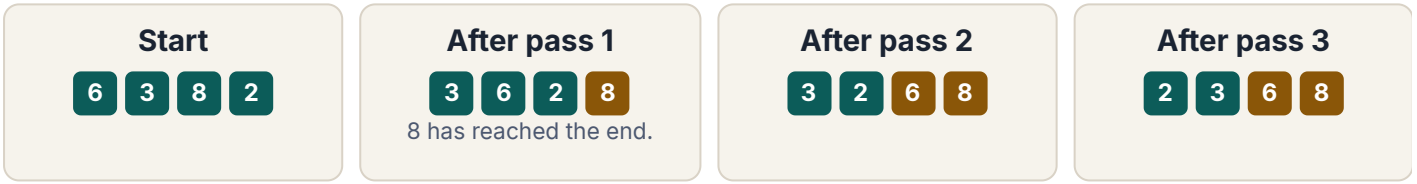
2.3 Logical reasoning inside a step

An IF is a claim. From “if the remainder on division by 2 is 0, then the number is even” you may trust the contrapositive: if it is not even, then the remainder was not 0. You may not automatically trust the converse unless the definition runs both ways. In this definition it does, because even means exactly that remainder. In a school rule such as “if the fee is unpaid, then the slip is withheld”, the converse (“if the slip is withheld, then the fee is unpaid”) may be false. Algorithmic thinking includes noticing which direction you used.

2.4 Bubble sort

Bubble sort, also called ripple sort, walks the list again and again, comparing neighbours and swapping them when they are out of order. After pass k, the k largest items that must bubble toward the end have arrived there. It sorts in place: the swaps happen inside the original list, so the extra memory is a single temporary variable. Equal items keep their original order if you swap only when the left one is strictly greater, so this version is stable.

```
DATA = [6, 3, 8, 2]
N = 4
for pass = 1 to N - 1
    for i = 1 to N - pass
        if DATA[i] > DATA[i + 1] then
            swap DATA[i] with DATA[i + 1]
        end if
    end for
end for
```



Pass 1 comparisons: $6 > 3$ swap, $6 > 8$ keep, $8 > 2$ swap, leaving 3, 6, 2, 8. Pass 2: $3 > 6$ keep, $6 > 2$ swap. Pass 3: $3 > 2$ swap. The list is 2, 3, 6, 8.

Cost

Neighbours compared, whether or not they swap: $\frac{n(n-1)}{2}$. For $n = 4$ that is 6. Double the list and the comparisons grow about fourfold, which is why we write the time as $O(n^2)$. Fine for a classroom list. Poor for a large register.

Why teach it

The picture is “two cards in the wrong order”. You can trace it on paper. It needs no second copy of the list.

Why not stop there

Every swap is three assignments. The nested loops do a lot of work even when the list is nearly ordered, unless you add an early-exit flag. Large data wants a faster sort.

2.5 Selection sort

Selection sort also uses nested passes, but it swaps less often. On pass k it finds the smallest item in the still-unsorted tail and swaps that item into position k.

```
for i = 1 to N - 1
    minPos = i
    for j = i + 1 to N
        if DATA[j] < DATA[minPos] then
            minPos = j
        end if
    end for
    swap DATA[i] with DATA[minPos]
end for
```

On 6, 3, 8, 2 the minimum of the whole list is 2, so pass 1 swaps it with 6 and leaves 2, 3, 8, 6. Pass 2 looks at 3, 8, 6 and finds that 3 is already home. Pass 3 swaps 8 and 6. Result: 2, 3, 6, 8. Comparisons are still about $n(n-1)/2$, so the time class is still $O(n^2)$, but the number of swaps is at most $n-1$. That matters when moving an item is expensive. Selection sort is not stable in the simple form above: an earlier equal value can be moved past a later one when the minimum is swapped into place.

2.6 Linear search

Linear search, or sequential search, looks at position 1, then 2, then 3, until it finds the target or finishes the list. The list may be unsorted. If the target sits at the end, or is absent, every element is examined.

```
found = false
position = 0
i = 1
while i ⩽ N and found = false
    if DATA[i] = target then
        found = true
        position = i
    else
        i = i + 1
    end if
end while
```

Search for 8 in 6, 3, 8, 2. Position 1 is 6, position 2 is 3, position 3 is 8. Stop. Three comparisons. Worst case comparisons: n, written $O(n)$. Doubling the list doubles the worst-case wait.

2.7 Binary search

Binary search is allowed only on a sorted list, stored so you can jump to the middle index. Arrays allow that. A pile of unsorted forms does not, until you sort it. Each step keeps the half that can still contain the target.

Middle $\mathbf{mid} = \mathbf{INT}\left(\dfrac{\mathbf{beg}+\mathbf{end}}{\mathbf{2}}\right)$. INT means the whole-number part. List positions are whole numbers, so $\mathbf{INT(3.5)} = \mathbf{3}$.

```
beg = 1
end = N
loc = 0
while beg ⩽ end and loc = 0
    mid = INT((beg + end) / 2)
    if DATA[mid] = target then
        loc = mid
    else if target < DATA[mid] then
        end = mid - 1
    else
        beg = mid + 1
    end if
end while
```

Sorted list 2, 3, 6, 8. Target 6. First middle is $\text{INT}((1+4)/2) = 2$, and position 2 holds 3. The target is greater, so beg becomes 3. Next middle is $\text{INT}((3+4)/2) = 3$, and position 3 holds 6. Found.

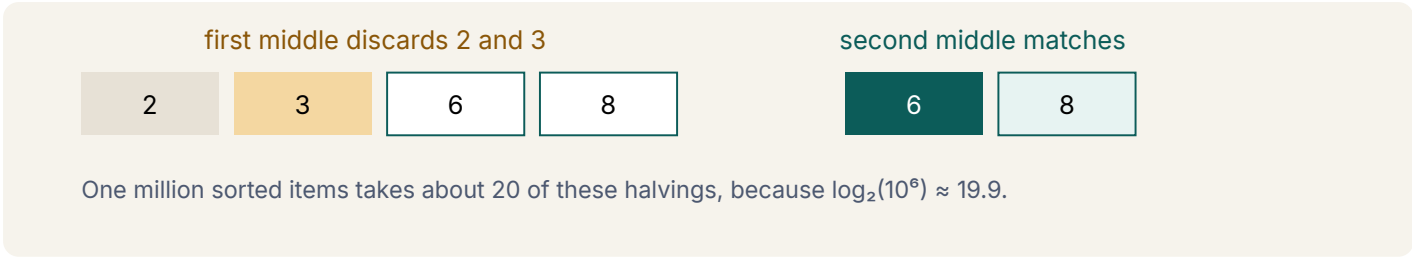


Figure 10. Each failed comparison throws away half of what remains.

Cost

Worst-case probes are about $\lfloor \log_2 n \rfloor + 1$, written $O(\log n)$. The price you pay first is sorting, plus careful index updates. An off-by-one on `beg` or `end` can loop forever or skip the target. Linear search is harder to get wrong.

2.8 Which algorithm to use

Situation	Choose	Reason
Short list, you must show every swap	Bubble sort	Clearest picture, stable if you swap only on a strict inequality
Swaps are costly, comparisons are cheap	Selection sort	At most $n - 1$ swaps
Unsorted, or searched once	Linear search	Sorting first would cost more than one scan
Sorted, and searched many times	Binary search	Sort once, then each lookup is logarithmic
Huge data in a real system	A faster sort than these two	$O(n^2)$ does not survive a large n. The course still expects you to trace the simple ones.

2.9 Correctness, efficiency, clarity

These are the evaluation criteria. A clever algorithm that fails one of them is not finished.

Correctness

For every allowed input, the output matches the definition. Include empty lists, one item, duplicates, and values that are not present. The procedure must end.

Efficiency

Time: how the step count grows with n. Space: extra memory beyond the input. Linear search is the right answer on a short unsorted list and the wrong answer on a million sorted records searched all day.

Clarity

Another student can trace it, name the variables, and change it without guessing. Pseudocode that hides a step inside “and so on” fails clarity even if the idea is correct.

A correctness trace. Input a whole number. If the remainder on division by 2 is 0, say even; otherwise say odd.

Input	Remainder	Branch	Output
3	1	else	odd
6	0	if	even
0	0	if	even
15	1	else	odd

The row for 0 is the one students skip. Zero is even. A trace that never leaves the happy path has not shown correctness.

Check questions — Unit 2

1. Name the five properties of a good algorithm, in words a non-programmer could follow.
2. Bubble-sort 5, 1, 4 on paper. How many neighbour comparisons does the formula predict?
3. Why is binary search refused a shuffled attendance list?
4. Give one input that would show an even/odd algorithm is wrong if it used “ends with 2, 4, 6, or 8” and forgot 0.

Programming Fundamentals

UNIT 3

By the end of this unit you can place Python among programming languages, find your way around Visual Studio Code, write variables, input, and operators correctly, choose sequence, selection, and repetition, call a library, and read a traceback.

3.1 Languages, and why this course uses Python

Machine language

Bits the processor executes. Exact, and painful to write by hand.

Assembly

Short mnemonics for those instructions. Still tied to one family of processors.

High-level

Closer to the problem than to the chip. Python sits here. An interpreter runs it, so you can edit and try a program quickly.

The same language shows up in software work, data analysis, machine learning, web services, automation, and security tooling. The reason it is the teaching language is narrower than the job list: the syntax is small, the feedback is immediate, and the ideas of Unit 2 (variables, conditions, loops) stay visible.

3.2 Visual Studio Code as the workbench

An IDE is the editor plus the tools around it. In VS Code the pieces you will actually touch are these.

Explorer

The folder of the program. Open a folder, not a random file lost on the desktop.

Search

Find a name across the folder when a program grows past one screen.

Source control

A history of edits. You are not required to master Git here; you should recognise the panel.

Run and Debug

Run the file, stop on a line, and inspect variables. The terminal is where `print` and input appear.

Extensions add a Python interpreter helper and a formatter. Install what the lab names. Do not collect extensions you cannot explain.

3.3 A first program, variables, and types

```
# A first program is allowed to be small
print("Hello, World!")
print("Welcome to Python")
print(100)
```

A variable is a name bound to a value. Names begin with a letter or underscore, may contain digits after that, and avoid keywords such as `if` and `for`. Python decides the type from the value you give it.

Type	Example literal	Use
`int`	`11`	Counts, roll numbers, whole marks
`float`	`3.5`	Measurements and money stored approximately
`str`	`"Ayesha"`	Text. Quotes are part of the syntax, not the text
`bool`	`True`, `False`	A condition that has already been decided

INPUT() `input()` always returns a string. `int(input("Age: "))` and `float(input("Price: "))` convert it. If you add a raw input to a number, Python raises an error instead of doing arithmetic.

```
name = input("Name: ")
marks = int(input("Marks out of 100: "))
print(f"Welcome, {name}. You scored {marks}.")
```

An f-string evaluates the braces and leaves the rest as text. `\n` starts a line. `\t` jumps a tab. `{total:.2f}` prints two digits after the decimal point.

3.4 Operators

An operator is the action. An operand is a value it acts on. In `10 + 20` the operands are 10 and 20.

Family	Symbols	A result worth remembering
Arithmetic	`+ - * / // % **`	`10 / 3` is about 3.333; `10 // 3` is 3; `10 % 3` is 1; `10 ** 3` is 1000
Assignment	`= += -= *= /= //= %= **=`	`a += 5` means `a = a + 5`. One equals sign stores a value
Comparison	`== != > < >= <=`	Two equals signs ask a question and produce True or False
Logical	`and or not`	`True and False` is False; `not True` is False
Membership	`in`, `not in`	`"P" in "Python"` is True

= AND == Inside a condition, `if a = 5` is a syntax error in Python. The test is `if a == 5`. The single symbol assigns.

3.5 Bitwise operators

Bitwise operators act on the binary digits, not on the decimal story you tell about the number. They are the programming form of the gates in Unit 1.

Shifts For a non-negative integer, $a \ll k = a \times 2^k$ and $a \gg k = \text{INT}(a/2^k)$. In two's complement, $\sim a = -a - 1$.

Take $a = 10$, which is 1010_2, and $b = 6$, which is 0110_2.

Expression	Bit picture	Decimal
$a \,\&\, b$	1010 AND 0110 = 0010	2
$a \mid b$	1010 OR 0110 = 1110	14
$a \oplus b$	1010 XOR 0110 = 1100	12
$a \ll 1$	1010 shifted left = 10100	20
$a \gg 1$	1010 shifted right = 0101	5

```
a = 10
b = 6
print(bin(a), bin(b))
print(a & b, a | b, a ^ b)
print(a << 1, a >> 1)
print(~a)   # -11
```

3.6 Control: sequence, selection, repetition

Every program in this unit is a mix of three structures. Sequence means the lines run from top to bottom. Selection means a condition chooses a branch. Repetition means a block runs while a condition holds, or once for each item of a sequence.

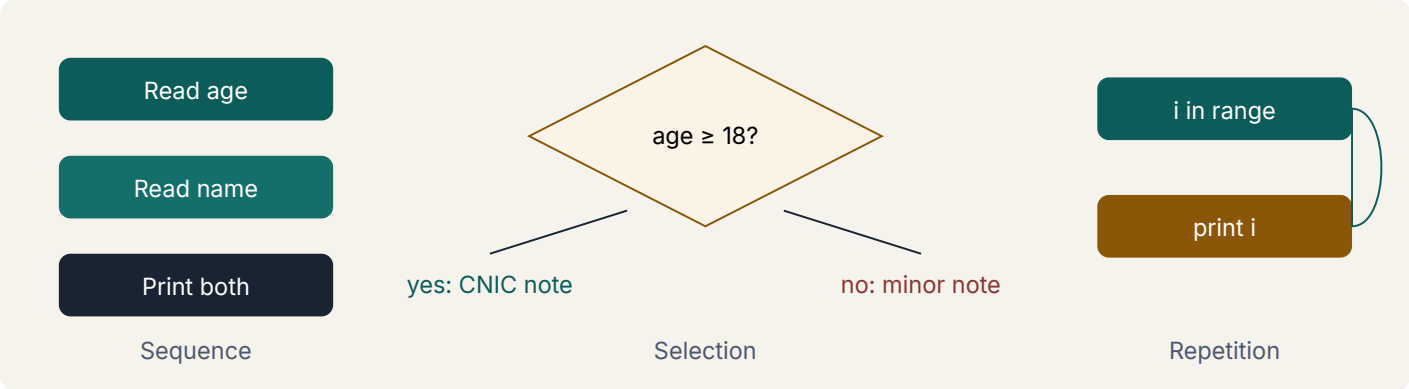


Figure 11. Three structures. Larger programs nest them; they do not invent a fourth.

```
age = int(input("Age: "))
if age ⩽ 0:
    print("Invalid age")
elif age ⩽ 12:
    print("Child")
elif age ⩽ 17:
    print("Teenager")
elif age ⩽ 59:
    print("Adult")
else:
    print("Senior")
```

Conditions are tried from the top. The first true branch wins, and the later ones are skipped. An `if` without `else` simply does nothing when the condition is false. Use `else` when every input must produce one of two messages. Nest an `if` only when the second question matters after the first has been answered, for example a fee check that runs only once age has already qualified.

for and range

`range(5)` yields 0, 1, 2, 3, 4. The stop value is excluded. `range(2, 8)` starts at 2. `range(0, 10, 2)` steps by 2. A negative step counts down: `range(10, 0, -2)` yields 10, 8, 6, 4, 2.

while

Use `while` when you cannot know the trip count in advance: read items until the user types `done`. Something inside the loop must change the condition. If nothing does, the program does not finish.

```
total = 0.0
count = 0
while True:
    item = input("Item, or done: ")
    if item.lower() == "done":
        break
    price = float(input("Price: "))
    if price ⩽ 0:
        print("Price must be positive")
        continue
    total += price
    count += 1
print(count, f"{total:.2f}")
```

`break` leaves the loop. `continue` skips the rest of this trip and starts the next test. A nested loop is a loop in the body of a loop. The inner one finishes its whole run before the outer one takes its next step. A 3 by 3 table of products is the clean demonstration:

```
for i in range(1, 4):
    for j in range(1, 4):
        print(i * j, end=" ")
    print()
```

3.7 A report-card calculation

Three subjects, each out of 100. Percentage is a ratio, then a scale.

Percentage $P = \dfrac{m_1 + m_2 + m_3}{300} \times 100$. Pass if $P \geq 50$. Change 50 only when the rule changes, and change it in one place.

```
m1 = float(input("Mathematics: "))
m2 = float(input("Science: "))
m3 = float(input("Computer: "))
total = m1 + m2 + m3
percentage = (total / 300) * 100
status = "PASS" if percentage ⩾ 50 else "FAIL"
print(f"Total {total:.0f}   {percentage:.2f}%   {status}")
```

3.8 Libraries

A library is code someone else has already tested. You import it instead of pasting it.

- `import math` then `math.sqrt(81)`, `math.floor`, `math.ceil`, `math.pi`.
- `import math as m` shortens the prefix.
- `from math import sqrt` lets you call `sqrt` with no prefix. Importing with `*` dumps every name into your file and makes clashes likely. Prefer the named form.
- `random.randint(1, 10)` includes both ends. `random.random()` is a float from 0 up to but not including 1.

A third-party library is installed into the environment, typically with `pip`, and only then imported. A built-in library such as `math` needs no install. If import fails, the message is almost always “not installed” or “misspelled”, not a bug in your arithmetic.

3.9 When the program stops

A traceback is a stack of calls. Read the last line first: that is the error type and the short reason. Above it, the file name and line number point at your statement. A syntax error means Python could not even start, often a missing colon, a bad indent, or `=` in a condition. A runtime error means the program started and then met an impossible operation.

```
ValueError: invalid literal for int() with base 10: 'abc'
```

That line came from `int("abc")` or from a user who typed letters into a numeric input. It does not mean every line above it is nonsense. An `IndentationError` means a block is not aligned with the header that owns it. Fix the reported line, run again, and deal with the next message. Guessing at distant lines creates new faults.

Check questions — Unit 3

1. What does `input()` return, and what do you wrap it in before you can test `age ⩾ 18`?
2. State the value of `17 // 5` and `17 % 5`.
3. Why can `range(1, 4)` never print 4?
4. A classmate writes `if percentage = 50`. Name the error and the repair.

Data and Analysis

UNIT 4

By the end of this unit you can separate data from information, name the parts of a database, choose keys, draw an entity-relationship diagram, protect referential integrity, and create a table, a form, and a query in Microsoft Access, including a grouped summary and a chart.

4.1 Data, information, and a DBMS

Data

Recorded facts that have not yet been given a purpose: 17, “Inaya”, “IS-101”, a date. On their own they do not tell you what to do.

Information

Data arranged so a person can decide. “Inaya is enrolled in IS-101 and has attended 17 of 20 classes” is information. The same digits, with a context and a question, changed jobs.

A database is an organised collection of related data. A database management system (DBMS) is the software that stores it and lets you define structure, insert and edit rows, ask questions, and produce reports, without each program inventing its own files. The advantages you should be able to list are less repeated data, consistency of shared facts, controlled access, and queries that survive after the person who typed the data has left.

4.2 The pieces, from small to large

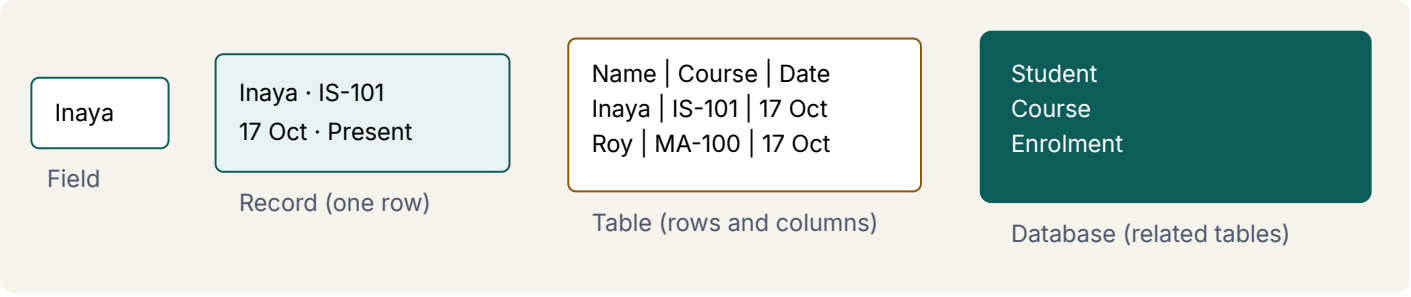


Figure 12. A field is one fact. A record is one entity’s facts. A table is the set of those records. A database is the tables that belong together.

4.3 Keys and integrity

Key	What it promises	Example
Candidate key	A field, or a combination, that could uniquely identify a row	StudentID, or a national identity number if it is unique and always present
Primary key	The candidate you actually chose. No duplicates, no empty value	StudentID
Composite key	A primary key made of more than one field	StudentID + CourseID in a small enrolment table
Foreign key	A field whose values must match a primary key in a parent table	Enrolment.StudentID pointing at Student.StudentID

Referential integrity is the rule behind the foreign key: every child value either matches a living parent or is deliberately empty, if empty is allowed. You cannot enrol student 999 if no student 999 exists. Cascade update rewrites the child when the parent key changes. Cascade delete removes the children when the parent row is removed, so a departed student does not leave orphan enrolments. Use cascade delete only when the child has no meaning without the parent. A book should not vanish because one borrower record was deleted.

4.4 The entity-relationship model

An ER model draws the world before you open Access. An entity is a rectangle: a type of thing, such as Student or Book. An attribute is an oval: one fact about that thing. A relationship is a diamond: a verb that connects entities. Underline the primary-key attribute.

Cardinality says how many. One-to-one: a person and that person’s current student card. One-to-many: one member borrows many records over the year, and each borrow record belongs to one member. Many-to-many: one member borrows many books, and one book is borrowed by many members over time. A relational schema cannot store a many-to-many link as a single foreign key. You resolve it with a new entity, the borrow record, and two one-to-many relationships.

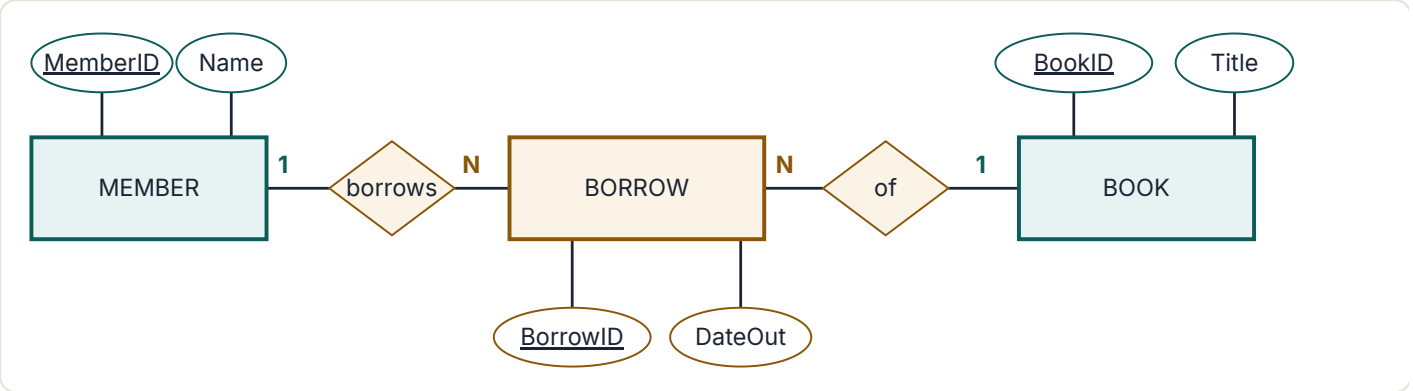


Figure 13. Member to Borrow is one-to-many. Book to Borrow is one-to-many. Together they replace Member-borrows-Book as many-to-many. Borrow stores MemberID and BookID as foreign keys.

The same steps work for any case study. Write the requirements in sentences. Underline the nouns: those are candidate entities. List attributes, mark primary keys, name the verbs, fix the cardinality, and only then resolve every many-to-many. A library needs at least Member (MemberID, name, contact), Book (BookID, title, author, publisher), and Borrow (BorrowID, MemberID, BookID, date out, date due). Author can stay an attribute while each book has one author string. It becomes an entity the day you must list every book by one author without retyping the name.

4.5 Objects you will meet in Access

Table

The stored rows. Design the table before you decorate anything else. A pretty form on a bad table preserves the bad table.

Form

A screen for one job: add a student, edit a phone number, avoid typing into the raw grid. Controls include text boxes, combo boxes, radio buttons, check boxes, dates, and command buttons.

Query

A question saved in the database: which students, which department, what average. Queries read tables. They are how analysis starts.

Report and chart

A layout for a reader who will not open the grid. Bar, column, and line charts belong here when the pattern is easier to see than the numbers.

4.6 Building a table

Plan on paper first: field name, data type, size, which field is the primary key, and which values are required. Then choose a view.

Datasheet view

Create → Table. You type into a grid, as in a spreadsheet. Access invents Field1, Field2 until you rename the headers. Fast for a sketch. Weak when you need validation rules, formats, and a carefully chosen key.

Design view

Create → Table Design. Each row of the designer is one field: name, data type, optional description. Select the key field and set Primary Key. This is the view to use once the plan exists.

Access type	Choose it for
Short Text	Names, course codes, email, phone numbers that will not be added
Number / AutoNumber	Ages, marks; AutoNumber when Access should issue the primary key
Date/Time	Date of birth, due date. Do not store a date as text if you will sort it
Currency	Fees and fines
Yes/No	A genuine two-way fact: fee paid, book on loan
Long Text	A remark longer than a short label

Data types you should be able to match to an example, not recite as a catalogue.

VALIDATION A required name, a rule such as `⩾ 0` on a mark, and a validation message the user can understand, all live in Design view. Datasheet view will not offer that control.

4.7 Forms

A form needs a record source: the table or query whose fields it edits. The Form Wizard asks which fields, which layout, and which title. Move StudentID, name, department, and contact across. Leave internal bookkeeping fields off the screen if the clerk should not type them. After the wizard, check tab order: the cursor should follow the way a person reads the paper form, not the order in which you dragged boxes.

4.8 Queries, filters, and summaries

Create → Query Design, add the Student table, and double-click the fields you want to see. A criterion under a column restricts rows. Text criteria are written in quotes.

One condition

Under Department, the criterion `"CS"` returns only that department. Under a mark column, `⩾ 50` keeps the passes.

AND

Two criteria on the *same* Criteria row must both be true: department CS and mark at least 50.

OR

The second condition goes on the *or* row, or you write `"CS" Or "IT"` in one cell. Either department qualifies.

A totals query groups rows. Open the Totals row. Group By the department. Use Count on the student id, Avg on the mark, Sum when the column is a quantity that should add, Min and Max when you need the extremes. The result has one row per department, which is the point: you have stopped listing students and started describing groups.

Average $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$. Access computes it inside the group you named. An average of all students in the college, and an average per department, answer different questions. The Group By column decides which one you asked.

Charts: a bar or column chart compares categories, such as average mark by department. A line chart follows a measure across time, such as loans per month. A stacked column shows parts inside a total, such as lectures by teacher inside one subject. Pick the chart from the question. A line through unrelated departments implies a time order that is not there.

Check questions — Unit 4

1. Why is a phone number usually Short Text rather than Number?
2. A borrow table stores BookID values that do not exist in the book table. Which rule has failed?
3. Member borrows Book is many-to-many. What new entity appears when you draw the final schema, and which foreign keys does it hold?
4. In Query Design, where do you type a second condition so that it means OR rather than AND?

Applications and Impacts of Computing

UNIT 5

By the end of this unit you can describe how AI, the Internet of Things, data analytics, and cloud services fit together, give a concrete use in cities, industry, health, education, energy, farming, and wearables, judge whether a source is primary, secondary, or tertiary, and argue a positive impact beside a negative one, including the role of assistive technology.

5.1 One pipeline, four names

A smart system is easier to remember as a pipeline than as a list of slogans. Devices sense the world. A network carries the readings. Storage, often on a cloud service, holds them. Analytics looks for a pattern. AI uses that pattern to choose an action. A person remains responsible for the choice when the action affects someone.

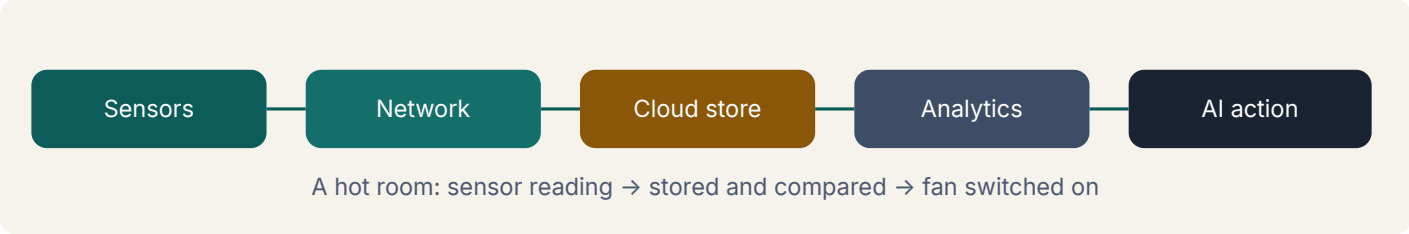


Figure 14. AI is the decision step, not the sensor and not the spreadsheet. Analytics is the study of the collected data. IoT is the connected sensing and acting.

Artificial intelligence

“Artificial” means people built it. “Intelligence” here means a task that used to need a judgement: recognising a face to unlock a phone, ranking videos, answering a routine question. The system is using data, not understanding a room the way a person does. Treat the output as a proposal.

Internet of Things

Ordinary objects with a sensor or actuator, an address, and a way to exchange data: watches, meters, cameras, pumps, vehicles. Common links include Wi-Fi, mobile data, and short-range radio. The object can report, and sometimes act, without a person typing the reading.

Data analytics

Collect, clean, group, and interpret. A teacher looking at a mark list to see which topic failed is doing analytics with a small set. A city doing the same with traffic sensors is the same idea at another scale. Sources include sensors, forms, transaction logs, and public records.

Cloud computing

Storage and software reached over the internet, rather than only on the device in your hand. Drive-style storage, a learning-management system, and shared classroom tools are the examples to cite. IoT uses the cloud because the devices themselves are too small to keep years of readings.

5.2 Where IoT shows up

Setting	What is sensed or shared	The decision it supports
Smart city	Traffic, air, bins, street lights	Change a signal plan, send a crew, dim a lamp
Industry	Machine temperature, vibration, output count	Service a motor before it stops the line
Healthcare	Pulse, ward occupancy, medicine cabinet	Alert a nurse; keep stock from running out
Education	Attendance devices, lab sensors, an LMS log	See who is present; share a resource
Home automation	Motion, temperature, a locked door	Light a hall, cool a room
Smart grid	Meter readings along the supply	Balance load and spot a loss
Smart farming	Soil moisture, local weather	Irrigate a dry plot and skip a wet one
Wearables	Steps, heart rate, sleep	A personal alert, not a diagnosis by itself

LOCAL CASE A canal farm that waters on a fixed clock wastes water on cool days and under-waters on hot ones. Soil sensors plus a weather feed let the pump run when the soil is actually dry. Name the sensor, the network, the place the readings are stored, and the action. That is a full IoT answer.

Careers sit on this pipeline: sensor technician, network installer, data analyst, junior AI developer, farm-technology adviser, health-informatics assistant. The skills that transfer are careful data collection, basic networking, a spreadsheet or a Python script, and the habit of asking what the device does when the link fails.

5.3 Information sources, and when they stop being reliable

Unit 6 will collect data. This unit classifies the sources you read.

	Primary	Secondary	Tertiary
What it is	Firsthand: the speech, the diary, the photograph, the original experiment, the official register	An account built from primary material: a news report, a textbook chapter, a review article	A guide to other sources: a directory, an encyclopedia entry, a bibliography
Strength	Closest to the event	Context, comparison, explanation	A fast map of where to look
Becomes unreliable when	The record is incomplete, altered, taken out of context, or the witness is guessing	The writer hides the sources, misquotes them, or writes far outside their knowledge	The entry is undated, anonymous, or points at sources you cannot check

An official identity register is a primary source for the facts it was created to hold. It is not a primary source for a rumour someone typed into a comment underneath a news story. A textbook is secondary: useful for learning, and the wrong citation if the question was “what did the agency itself publish?”. Reliability is not a badge that sticks forever. Check who produced it, when, for what purpose, and whether you can see the underlying record.

The matching skills are library and archive work, fact-checking, research assistance, and science communication. Each of them is mostly the discipline of saying where a claim came from.

5.4 Impacts, including assistive technology

Gains you can evidence

- A bank customer reaches an account without travelling to a branch.
- A student replays a lesson; a clinic keeps a record that another shift can read.
- A farmer skips a wasted irrigation cycle.

Costs you must also name

- Dependence: when the system stops, the manual skill has already faded.
- Privacy: a wearable and a school camera collect more than the user notices.
- Uneven access: a service that exists only on a new phone excludes people.
- Unchecked automation: a wrong sensor still triggers a confident action.

Assistive technology is the family of tools that reduce a barrier: mobility aids, screen readers and captions, hearing support, communication devices, and accessible interfaces in classrooms and hospitals. The purpose is participation. A student who can hear the lesson through captions, or read it through a screen reader, is using computing as access rather than as convenience. Careers in this area need both the technical skill and the habit of testing with the person who will rely on the tool.

A FAIR PARAGRAPH One positive, one negative, and the condition that decides which one you are living in. “Mobile banking saves a journey; it also fails the customer who has no signal and no alternative counter.” That is an analytical answer. A list of brand names is not.

Check questions — Unit 5

1. In a smart home that turns on a fan, which part is IoT, which part is analytics, and which part is AI?
2. Give one reason a primary source can still be a bad source for a school project.
3. Name one assistive technology and the barrier it is meant to lower.
4. Why does a soil sensor belong in an answer about farming, while “technology helps farmers” does not?

Digital Literacy

UNIT 6

By the end of this unit you can run a precise search, design a way to collect original data, tell primary data from secondary data, present both with a suitable tool, and publish a digital artefact that cites its sources and treats respondents fairly.

6.1 Literacy, not just access

Digital literacy is the ability to find, judge, shape, and share information with digital tools, and to do it safely. Owning a phone is not the skill. The skill is knowing which query to type, which result to trust, how to collect a fact you cannot look up, and how to show your work so someone else can check it.

The information age is the condition in which that skill matters: a large volume of data is published continuously, and access is no longer the scarce part. Judgement is.

6.2 Qualitative and quantitative data

Qualitative

Words about experience, reason, and meaning. “I revise with recorded lectures because the classroom is noisy.” You group comments; you do not average them.

Quantitative

Numbers you can count or measure. Hours online, a score out of 10, a yes that you later total. You can compute a mean only after the question was asked as a number.

	Qualitative	Quantitative
Question shape	How, why, what is it like	How many, how often, how much
Typical tool	Interview notes, open responses	Counts, scales, spreadsheet columns
Weakness	Harder to compare fairly	Misses the reason behind the number

A strong class inquiry uses both. “Average phone use is 3.5 hours” is incomplete until a comment explains whether those hours were lectures or a feed.

6.3 Ways of collecting

Interview

Few people, deep answers, a prepared guide, permission to note or record. Good for reasons. Poor for claiming “most students”.

Survey

The same questions put to many people, on paper or in a form tool. Good for counts. Weak if the questions nudge the answer.

Observation

Watch a real activity and record an agreed checklist. Good for what people do. Weak if they change behaviour because they are watched.

Prototype

A small version of a design, tested with users. The data is what failed in the trial, not a speech about the idea.

Simulation

A model of a system you cannot safely or cheaply run, such as a queue at a lab or a spread of a rumour through a class network. Say what the model assumes. A simulation reports the model, not the world, until you compare it with a real measurement.

6.4 Primary data and secondary data

This distinction is about *who collected it*, which is not the same phrase as the primary and secondary *sources* of Unit 5, though they often line up.

	Primary data	Secondary data
Definition	Collected by you for this question: your survey, interview, observation, or experiment	Collected by someone else, then reused: a report, a book, a published table, a database
Advantage	Fits your question and your class	Wider than one classroom; already cleaned
Risk	A small or biased sample	A different definition, an old year, or a hidden agenda

DO NOT SWAP THEM A government PDF you downloaded is secondary data for your project, even if it was primary work for the agency that gathered it. Your classmate's survey responses are primary data for you. Copying those numbers out of a website does not make them primary.

6.5 Designing the approach

Write these down before you collect anything. A project that starts with a form and invents the question afterwards cannot be repaired by a better chart.

1. **Question.** One sentence a stranger can answer with yes, no, or a measurement. "How does internet use affect weekday study time in this Class XI section?" can be studied. "Is the internet good?" cannot.
2. **What you need.** Mark each fact as qualitative or quantitative, primary or secondary.
3. **Who.** The population, then the people you will actually reach. A convenience sample of friends is allowed in class if you name it as convenience and refuse to generalise to the country.
4. **Method and tool.** Survey, interview, observation, prototype, or simulation, plus the form, sheet, or notebook.
5. **Ethics.** Ask permission. Collect only what the question needs. Do not publish names beside sensitive answers. Let a respondent stop.
6. **Analysis plan.** Which total, which average, which groups of comments, which chart.

6.6 Searching like a researcher

A handful of results is a failure if they are the wrong handful. Operators narrow the pile.

Operator	Effect	Example
Quotation marks	The words stay in that order	`"digital literacy" survey`
`filetype:`	A format, useful when you want a report	`"internet usage in Pakistan" filetype:pdf`
`site:`	One site or domain	`attendance policy site:.edu.pk`
AND, OR, NOT / minus	Require, allow alternatives, or exclude	`smartphone study OR homework -sale`

Then judge the page you opened. Who wrote it, what date it carries, why it was published, whether it cites a method, and whether a second independent source agrees with the number. A recent date does not repair a missing author. An attractive layout does not repair a missing method.

6.7 Presenting

Spreadsheet

The working table: raw responses, then calculated columns. Average, count, and a chart that uses one question. Keep the raw sheet. A chart without the table cannot be checked.

Slides

One claim per slide, a chart that supports it, and very little prose. The talk carries the sentences the slide does not.

Infographic

A single page for a reader who will not study the sheet. Few numbers, each one labelled with its unit and its sample size.

Report

The full argument: question, method, results, limits, sources. This is the artefact a teacher can mark against the outcomes.

From a sheet If five students report 2, 3, 3, 4, and 3 hours, then $n = 5$ and $\bar{x} = (2 + 3 + 3 + 4 + 3)/5 = 3$. State n beside every average. An average of three willing friends is not an average of the section.

6.8 A worked inquiry

Question: in this section, how is weekday phone use split between study and distraction? Secondary search first, with a quoted phrase and `filetype:pdf`, so the introduction is not only opinion. Then a short survey of the section.

Item	Kind
On a weekday, how many hours is the phone on for study?	Quantitative
How many hours is it on for something else?	Quantitative
What do you open most when you meant to study?	Quantitative if the choices are fixed; qualitative if it is free text
Describe one way the phone helps a piece of homework.	Qualitative
Describe one way it pulls you off the homework.	Qualitative

Each item has one job. Mixing a reason into a yes/no question makes the column useless later.

Enter the numbers in a sheet. Compute the two averages and the count of each “opened most” choice. Make two visuals only: a bar chart of the activities, and a comparison of the two hour-averages. In the comments, group the helping remarks and the distracting remarks rather than pasting every sentence. Put one secondary finding next to yours and say whether it agrees. End with a recommendation the section can actually try, and a sentence on the limit of the sample.

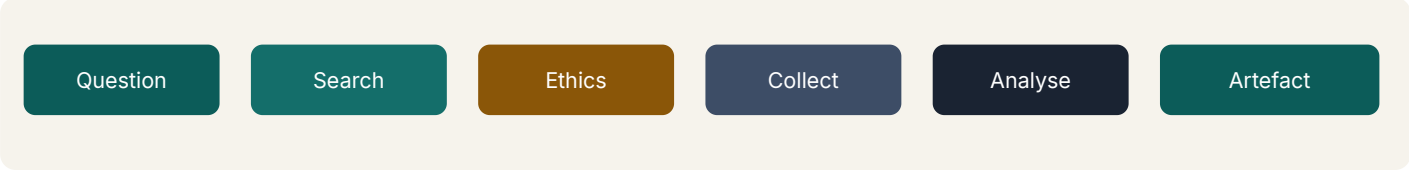


Figure 15. Ethics sits before collection, not in a footnote after the survey has already been pushed into a group chat.

6.9 The artefact, the credit, the limit

Whatever form you hand in, the reader should find, in this order: title, research question, short introduction, method, primary findings, secondary summary, the charts, a conclusion, a recommendation, and references. Decoration that does not carry one of those jobs makes the artefact worse.

Plagiarism is presenting someone else’s words, images, numbers, or structure as if you made them. A citation is the repair for honest reuse: author, title, year, and where you found it, in the format your teacher set. Paraphrase still needs a citation, because the finding was not yours. Ethics of collection and ethics of credit are the same habit: do not take from a person, or from an author, without saying so.

Check questions — Unit 6

1. Which query is better for a citable report, and why: `internet usage Pakistan` or `"internet usage in Pakistan" filetype:pdf`?
2. A student copies a table from a ministry website into the project. Is that table primary data or secondary data in the project?
3. Write one survey item that collects quantitative data and one that collects qualitative data on the same topic.
4. Name the ten parts of a finished digital artefact, in an order a reader can follow.

Answer key

CHECK QUESTIONS

Compare your working with these notes after you have written an answer. A matching word is less important than a matching reason.

Unit 1

1. Temperature is continuous: between two readings another reading is possible. The student count is discrete: it jumps by whole people.
2. Inside the bar first: $A + B = 1$, so $\overline{A + B} = 0$. Then $0 \cdot 1 = 0$. NOT applies to the whole sum because the bar covers the bracket.
3. NAND. $Y = \overline{A \cdot B}$. The output column is 1, 1, 1, 0.
4. Adjacent columns must differ by one bit so that a grouped pair drops exactly one variable. Ordinary binary puts 01 beside 10, and those codes differ in both bits.
5. Waterfall for the library system with stable, listable requirements. Agile (Scrum) for the phone app whose users will revise the backlog after they try a sprint.
6. UDP, at the transport layer (OSI layer 4, TCP/IP transport).

Unit 2

1. Input, output, definiteness, finiteness, effectiveness. Then judge with correctness, efficiency, and clarity.
2. Full nested loops, no early exit: $\frac{3 \cdot 2}{2} = 3$ neighbour comparisons. Pass 1 does two (5 with 1, then 5 with 4) and leaves 1, 4, 5. Pass 2 does one (1 with 4) and keeps the order.
3. The middle-index test is meaningful only when everything left of the middle is smaller and everything right is larger. A shuffled list does not offer that promise.
4. The input 0, or 10, or any even number that does not end in 2, 4, 6, or 8. The remainder test classifies 0 as even; the last-digit list as stated does not.

Unit 3

1. A string. Wrap it with `int(...)` before a numeric comparison.
2. Floor division gives 3. Remainder gives 2.
3. The stop value is excluded. The values are 1, 2, and 3.
4. A single equals sign assigns; a condition needs $=\!=$. Python reports a syntax error. Write `if percentage == 50`, or $\geqslant$ if the rule is a threshold rather than an exact mark.

Unit 4

1. Leading zeros matter, you will not add two phone numbers, and symbols such as a leading plus are text. Number types are for arithmetic.
2. Referential integrity. BookID in the child table is a foreign key with no matching parent.
3. A borrow (loan) entity. It holds MemberID and BookID as foreign keys, plus its own primary key and the dates of the loan.
4. On the separate *or* row of the design grid, or with the word Or inside one criterion. A second criterion on the same Criteria row means AND.

Unit 5

1. The sensor and the fan link are IoT. Comparing the reading with a threshold is analytics. Choosing to switch the fan on is the AI step in the course’s picture of a smart system. You may add that a fixed threshold is a very small decision rule.
2. It can be incomplete, edited, out of context, or simply one person’s unchecked note. Firsthand is not the same as true.

3. A screen reader lowers the barrier of printed text for a blind student; captions lower the barrier of spoken audio for a deaf student. Name the barrier, not only the gadget.
4. The sensor names the data, the action, and the decision. The slogan names none of them, so it cannot be tested or designed.

Unit 6

1. The second. Quotes keep the phrase intact and `filetype:pdf` prefers a document you can cite. The first query also matches ads, shops, and loose pages.
2. Secondary data in your project: you did not collect it for this question. It may have been primary for the ministry.
3. Quantitative: “How many hours did you study yesterday?” Qualitative: “What made yesterday’s study session hard?” Other pairs are fine if one yields a number and the other yields a description.
4. Title, research question, introduction, method, primary findings, secondary summary, charts, conclusion, recommendations, references.

Formula strip for the last week

Rows	2^n combinations in a truth table of n inputs.
Order	Parentheses, NOT, AND, OR. $\overline{A \cdot B} = \overline{A} + \overline{B}$.
XOR	$A \oplus B = A\overline{B} + \overline{A}B$.
Sort	Neighbour comparisons in the simple bubble sort: $\frac{n(n-1)}{2}$, time $O(n^2)$.
Search	Linear worst case n. Binary, on a sorted list: $\mathbf{mid} = \mathbf{INT}((\mathbf{beg} + \mathbf{end})/2)$, about $\lfloor \log_2 n \rfloor + 1$ probes.
Shifts	$a \ll k = a \times 2^k$ and $\sim a = -a - 1$ in two’s complement.
Percent	$P = (\mathbf{total}/\mathbf{maximum}) \times 100$.
Mean	$\bar{x} = \frac{1}{n}\sum x_i$, and n is part of the answer.

End of the lecture notes. Trace one example from each unit in your own handwriting before the test: a gate diagram, one sort pass, one Python branch, one foreign key, one IoT pipeline, and one search query.